

Chapter 6. Schemas

This chapter covers

- Event schemas and schema technologies
- Representing events in Apache Avro
- Self-describing events
- Schema registries

In the first part of this book, we took a wide-ranging look at event streams and the unified log, using fictitious online retailer Nile. We looked in depth at adding a unified log to our organization and experimented with different stream-processing frameworks to work with the events in our Kafka topics.

But like fast-food addicts, we didn't spend a lot of time thinking about the *quality* of the events that we were feeding into our unified log. This part of the book aims to change this, by looking much more closely at the way we model the events flowing through our unified log, using *schemas*.

Working for Plum, a fictitious global consumer-electronics manufacturer, we will introduce Plum's first event, a regular health-check "ping" emitted from each NCX-10 machine on the factory floor. Like every unified log, Plum's is fundamentally decoupled: consumers and producers of event streams have no particular knowledge of each other. This puts the onus on event schemas to serve as the contract between Plum's event consumers and producers.

How should we define Plum's event schemas? The chapter introduces four widely used schema technologies: Apache Avro, JSON Schema, Apache Thrift, and Google's protocol buffers. More than just data serialization systems, schema technologies like Avro offer schema evolution support, generation of bindings for your programming language, and multiple options for encoding the events on disk.

Adopting Avro for Plum, we will model the NCX-10 health-check event in the Avro schema language, and then autogenerate Plain Old Java Object (POJO) bindings for that event in Java. We'll then test this with a simple Java app that deserializes a JSON-format Avro health-check event, prints it out as a POJO, and then reserializes it into Avro's binary format.

With simple Avro parsing under our belt, we'll continue into design questions around associating events with their schemas. We suggest a couple

of possible approaches to making this event-schema association, before arguing strongly for *self-describing* events. These are events that include a metadata “envelope,” referencing the schema, alongside the event itself. To work with these events, you first read the event’s outer envelope to discover the event’s Avro schema, and then retrieve the schema and use it to deserialize the event’s data payload itself.

Before we know it, we will be proliferating schemas for our employer Plum. And these have to live somewhere—namely, in a *schema registry*. We will wrap up the chapter with a brief look at the core attributes of a schema registry, before introducing the two most widely used schema registries: Confluent Schema Registry and Snowplow’s own Iglu schema registry. These two registries are broadly similar but have interesting differences of design decision, which we’ll look at last.

Let’s get started!

6.1. An introduction to schemas

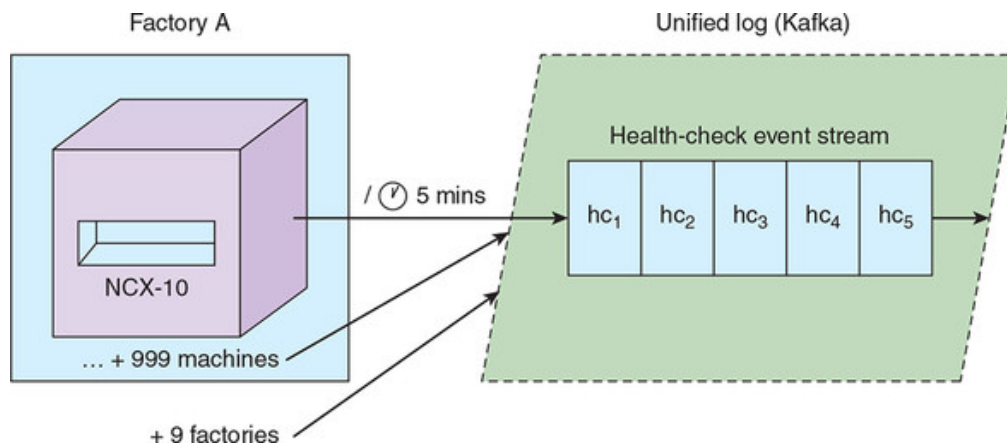
To start working with schemas, we’ll first need some events. For this part of the book, we’ll leave Nile behind and work with another fictitious company with a unified log, called Plum. Let’s introduce Plum and its event streams.

6.1.1. Introducing Plum

Let’s imagine that we are in the Business Intelligence (BI) team at Plum, a global consumer-electronics manufacturer. Plum is in the process of implementing a unified log. For unimportant reasons, its unified log is a hybrid, using Amazon Kinesis for certain streams and Apache Kafka elsewhere; Amazon Kinesis (<https://aws.amazon.com/kinesis/>) is a hosted unified log service available as part of Amazon Web Services. In reality, using both Kafka and Kinesis is a fairly unusual setup, but it has the advantage for us that we can work with both Kinesis and Kafka in this section of the book.

At the heart of the Plum production line is the NCX-10 machine, which stamps new widgets out of a single block of steel. Plum has 1,000 of these machines in each of its 10 factories. Our corporate overlords at Plum want us to program each machine to emit key metrics in the form of a health check to a Kinesis stream every second, as shown in [figure 6.1](#). A Kinesis stream is similar to a Kafka topic (don’t worry, we’ll explore Kinesis in detail in the next chapter).

Figure 6.1. All of the NCX-10 machines in Plum’s factories should emit a standard health-check event to a Kinesis stream every second.



Across Plum’s 10 factories, we have 10,000 machines. With each machine emitting a health check every second, that’s 36 million health checks landing in the Kinesis stream each hour, a substantial first event source for Plum’s unified log.

We sit down with the plant maintenance team at Plum to find out what health-check information we can retrieve from the NCX-10 machines. We discover a few interesting data points:

- The name of the factory that the machine is installed in.
- The machine’s serial number, which is a string.
- The machine’s current status, which can be one of `STARTING`, `RUNNING`, or `SHUTTING_DOWN`.
- The time when the machine was last started, which is a Unix time-stamp accurate to milliseconds.
- The machine’s current temperature, in Celsius.
- Whether or not the machine has been earmarked as being end-of-life, meaning that it should be scrapped soon.
- If the factory has multiple floors, the machine knows which floor it is situated on.

In [part 1](#) of this book, we used JSON as a way of encoding all of Nile’s on-line shopping-related events. Channeling this expertise for Plum, we can quickly sketch out an example JSON instance representing a hypothetical health check from one of the NCX-10 machines:

```
{ "factory": "Factory A",  
  "serialNumber": "EU3571",  
  "status": "RUNNING",  
  "lastStartedAt": 1539598697944,  
  "temperature": 34.56,  
  "endOfLife": false,
```

```
"floorNumber": 2
}
```

The plant engineers squint at the preceding example JSON and confirm that this is pretty much everything that an NCX-10 machine can usefully emit.

This JSON is a great start. If this were still [part 1](#) of the book, we would take this JSON format and run with it, designing and building stream processing applications to read and write these JSON-based events. But this is [part 2](#), and for Plum we can go further: we are going to replace this JSON format with a formal *schema* for our health-check event. Before we can do this, you need to understand exactly what a schema is, and why it is so useful.

6.1.2. Event schemas as contracts

Remember that the powers-that-be at Plum have so far given us a narrow brief: programming each of the NCX-10 machines to emit a regular health-check event. We don't yet know how Plum wants to use this event, or even which teams within Plum will be tasked with working with these events in Plum's unified log.

This is a common occurrence: in the real world, someone else, maybe even a different team or department or country, will most often consume the event stream that you create. A unified log is a fundamentally *decoupled* architecture: consumers and producers of event streams have no particular knowledge of each other. This is in stark contrast to a traditional single monolithic software project, where you can see all parts of the code evolving in lockstep within source control, with a compiler and test suite constantly enforcing the integrity of the whole system.^{[\[1\]](#)}

[1](#)

You can find more information about continuous integration at Wikipedia: https://en.wikipedia.org/wiki/Continuous_integration.

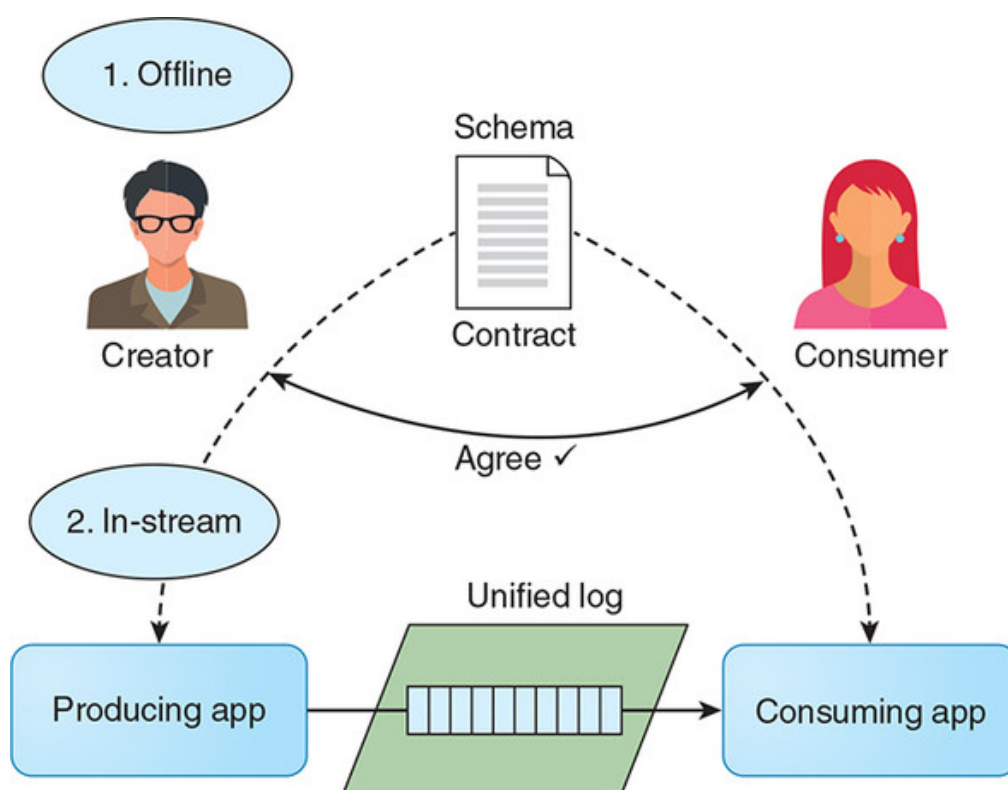
If you think about it, the decoupled unified log approach is analogous to the widgets that Plum's beloved NCX-10 machines are churning out. Plum doesn't necessarily know which companies will buy the widgets, or what purposes those customers will put the widgets to. How does Plum guarantee that its widgets are fit-for-purpose for its customers? They do this by signing contracts with their customers that dictate certain standards that their widgets will conform to. Perhaps the widgets will always be SAE steel grade 12L14, pass the GS quality and safety test, and use the Unified

Thread Standard for its screw threads. With these standards enforced in contracts, Plum’s customers can be confident that they can build products that make use of Plum’s widgets.

The consumers of the event streams within our unified log need exactly the same kind of guarantees as the buyers of Plum’s widgets. We can give them these guarantees by using *schemas* for the events we are storing in our unified log. *Schema* is the Greek word for *shape*, and an event schema is simply that: a declaration that some set of events in our unified log will follow a predefined shape.

In the absence of formal integrations between systems, our event schemas are the closest we come to a *contract* between the system generating an event stream and the systems consuming that stream. [Figure 6.2](#) illustrates this contract.

Figure 6.2. The producer of the event agrees to the schema of the event with the initial known consumer of the event. This acts as a contract between both parties. Then, in the stream, the producing app creates events using the agreed-upon schema, and the consuming app can happily read those events, safe in the knowledge that the events will conform to the schema.



By agreeing to the schema up-front and writing our code in a way that ensures that the events we generate conform to that schema, we can avoid a huge amount of pain for whoever is consuming those events later. But how do we represent our event schemas? The good news is that we have a choice of a variety of schema technologies, often called *data serializa-*

tion systems, which have emerged over the past few years. Before we introduce these, let's briefly cover the capabilities of these technologies.

6.1.3. Capabilities of schema technologies

In [part 1](#), we built applications that produced and consumed events that were represented using plain JSON. The events we created for online retailer Nile certainly each had a *shape*, but that shape was not formally documented in a machine-readable way; we could say that Nile's events had *implicit* schemas rather than explicit.

To be clear, JSON is a data serialization system, but it's not one that allows us to provide the kinds of contracts that we need for our unified log. The schema technologies that we will look at in the next section all provide a *schema language* to precisely define the data types of the individual properties of your business entities.

If you have ever worked with a *strongly typed* programming language, you will be familiar with data types. Simple data types such as string or integer are almost always well represented by these schema languages; different schema technologies will, however, have varying support for the following:

- *Compound data types* such as arrays and records or objects
- *Less common data types* such as timestamps, UUIDs, and geographical coordinates

Going further, the schema technologies we will look at all additionally exhibit at least some of these six capabilities (the abbreviations in parentheses refer to [table 6.1](#) in the following subsection):

- *Multiple schema languages* (LNG)—Some schema technologies provide multiple ways in which you can express the data types for your business entities. For example, you may be able to write your schemas declaratively in JSON, or there may some form of interface description language (IDL) with a more C-like or Java-like syntax.
- *Validation rules* (VLD)—Some schema technologies go further than data types, by letting you express validation rules (sometimes called *contracts*) on the event's properties. For example, you might express that a longitude is not just a floating-point number, but a floating-point number that can't be more than 180 or less than -180, and that latitude must be no more than 90 or less than -90.
- *Code generation* (GEN)—Whatever syntax the schema is expressed in, we are likely to want to also interact with events represented by the schema in code we write (for example, our stream processing apps).

To facilitate this, schema technologies often support code generation, which will generate idiomatic classes or records in your preferred language from the schema.

- *Multiple encodings* (ENC)—Some schema technologies support multiple encodings of the data, often a compact binary format and a human-readable format (perhaps JSON-based).
- *Schema evolution* (EVO)—The properties in our Plum events will likely evolve over time. Some of the more sophisticated schema technologies have built-in support for schema evolution, which makes it easier to consume different versions of the same schema.
- *Remote procedure calls* (RPC)—This is not a feature that we need, but some data serialization systems also come with a mechanism for building remote procedure calls, distributed functions that use data types for expressing the functions’ arguments and return values.

Don’t worry if some of these capabilities sound a little abstract right now. The rest of this chapter will use Plum’s business requirements to make all this much more concrete.

6.1.4. Some schema technologies

A variety of schema technologies, also known as *data serialization systems*, have emerged over the past few years, each with subtly different capabilities. [Table 6.1](#) introduces four of the most widely used systems and lays out their design relative to the six capabilities introduced in the preceding section.

Table 6.1. Examples of schema technologies

Schema tech	LNG	VLD	GEN	ENC	EVO	RPC
Apache Avro	JSON, IDL	No	Yes	Binary, JSON	Yes	Yes
Apache Thrift	IDL	No	Yes	Five encodings	Yes	Yes
JSON Schema	JSON	Yes	No	JSON	No	No
Protocol buffers	IDL	No	Yes	Binary, JSON	No	Yes (gRPC)

Let's go through each of these schema technologies briefly.

Apache Avro

Apache Avro (<https://avro.apache.org/>) is an RPC and data serialization system that was developed as part of the Apache Hadoop project, and shares many of the same authors, including Hadoop pioneer Doug Cutting.

Avro has a declarative JSON-based schema language for describing data types, as well as an alternative language, called Avro IDL, which is more C-like. Avro has two representations for data: a compact binary encoding and a human-readable JSON encoding; the latter follows a few additional rules to represent Avro features that are not natively supported in JSON. The binary encoding is used much more widely than the JSON encoding, and as such the tooling for the binary encoding tends to be more fully featured.

As a software engineer, you can interact with Avro in one of two ways. The first is to use code generation ahead of time, creating representations for the Avro data types in your preferred programming language; you can then round-trip the Avro-encoded data to classes or records in your code, and back again. The second approach involves using the schema at run-time to parse the data in a generic way; this is a good fit for dynamic languages and for situations (common in a unified log) in which the record types to process are not known ahead of time.

With its origins in the Hadoop project, Avro was designed from the outset for use in data processing, and as a result has a sophisticated take on schema evolution. When a consuming application is reading an Avro record, it must have a copy of the schema that was used to write the record, but it can also supply another version of that same schema; these are the *writer's schema* and the *reader's schema*, respectively. Avro will then transparently use resolution rules between the two schema versions to represent the data in the reader's schema. This allows an application to happily process an archive of events written by multiple historic versions of a given schema.

Apache Thrift

Apache Thrift (<https://thrift.apache.org/>) is pitched as a framework for cross-language services development. As part of this, it includes a definition language that allows you to define data types as well as service interfaces. One of the biggest selling points is its broad language support:

Thrift has first-class support for C++, Java, Python, PHP, Ruby, Go, Erlang, Perl, Haskell, C#, Cocoa, JavaScript, Node.js, Smalltalk, OCaml, and Delphi.

Unlike the other schema technologies, Thrift is much less opinionated about how the data should be encoded: it offers three binary encodings and two JSON-based encodings. When you define a schema in Thrift, you manually assign each property a tag (1, 2, 3, and so forth), and these tags are stored in the encoded data along with the data type; this is how Thrift supports schema evolution: an unknown property can be skipped, or a re-named property correctly identified.

Thrift was created at Facebook. The story goes that “Xooglers” created it because they missed using Google’s own protocol buffers, which hadn’t been open sourced at that point.

JSON Schema

JSON Schema (<https://json-schema.org/>) is slightly different from the other schema technologies described here. JSON Schema is a declarative language, itself written in JSON, for describing your JSON data’s format; it is easily written by humans and makes it possible for computers to validate and parse individual JSON data. If you are familiar with XML, the relationship between JSON and JSON Schema is not dissimilar to that between XML and its document type definitions (DTDs).

JSON Schema doesn’t concern itself with data serialization at all; it’s simply an overlay for defining the shape of JSON data. It also doesn’t depend on (or offer) code generation, although there are community-led initiatives to implement code generation for various languages.

Although not as fully featured as Avro, JSON Schema does have two distinct strengths:

- ***Rich validation rules***— It’s possible to express sophisticated data validation rules in JSON Schema, including minima and maxima for numbers, and regular expressions for strings. Used creatively, this allows you to define your own simple data types.
- ***It’s just JSON***— Because JSON Schema is simply a schema overlay over plain JSON, you don’t have to rewrite your systems to use another data serialization system. If you are comfortable with JSON, JSON Schema is quick to pick up. You can also backfill schemas for existing JSON data after the fact by using tools such as Schema Guru (<https://github.com/snowplow/schema-guru>).

Protocol Buffers

Protocol buffers (<https://developers.google.com/protocol-buffers/>) are a schema technology and data serialization mechanism from Google, currently on its third major version (and open sourced from its second major version). Protocol buffers are similar to Thrift; they support a protocol definition syntax that lets the user define data structures in .proto files.

Data in protocol buffers is serialized into a binary format; there is also a self-describing ASCII format, but this does not support schema evolution. As with Thrift, integers are used to tag each property within the data structure and to handle schema evolution. One neat aspect of protocol buffers is that arrays are represented with the `repeated` modifier, and the same encoding is used for repeated, optional, and required properties, making it possible to easily migrate a property from being, for example, optional to being an array.

Protocol buffers can be used as a standalone schema technology, but they are also closely associated with and used with Google's RPC framework, called gRPC.

This concludes our whirlwind introduction to four major schema technologies—but how do we choose between these for our bosses at Plum?

6.1.5. Choosing a schema technology for Plum

Choosing our schema technology is one of the most important decisions we will make for our unified log at Plum. We can adopt different stream-processing frameworks, but we get to adopt only one schema technology. All of the events we write into our unified log will be stored with these schemas; the archive of events from Plum's unified log (which hopefully will stretch into many years) will be represented in this format too.

Avro, JSON Schema, and protocol buffers are all growing in popularity; interest in Thrift is quite possibly growing too. So how do we choose between them? Here are simple rules that might help:

- If you are using or plan to use gRPC, consider using protocol buffers for your unified log.
- Likewise, if you are using Thrift RPC already, consider using Thrift.
- If you have existing batch- or stream-processing systems that make heavy use of JSON, or you expect a lot of event authoring to be done by developers who prefer JSON, consider JSON Schema.
- Otherwise, use Avro.

In the case of Plum, none of the first three rules are met, so we are going to go with Avro. Enough theory—let's get started.

6.2. Modeling our event in Avro

From our colleagues on the factory floor, we know what data points our health-check events need to contain, and from the brief review in the previous section, we know that we want to use Avro as our schema technology at Plum. So, we are ready now to model our health-check event in Avro.

6.2.1. Setting up a development harness

Avro schemas can be defined in plain JSON or Avro IDL text files, but the schema file needs to live somewhere, so we'll create a simple Java app to hold it, called `SchemaApp`. This will give us a harness to experiment with Avro's code-generation capabilities, as well as to explore its encodings.

We are going to write the harness app by using Gradle. First, create a directory called `plum`, and then switch to that directory and run the following:

```
$ gradle init --type java-library
...
BUILD SUCCESSFUL
...
```

Gradle has created a skeleton project in that directory, containing a couple of Java source files for stub classes called `Library.java` and `LibraryTest.java`. Delete these two files, because we will be writing our own code shortly.

Next let's prepare our Gradle project build file. Edit the file `build.gradle` and replace its current contents with the following listing.

Listing 6.1. `build.gradle`

```
plugins {
    id "com.commercehub.gradle.plugin.avro" version "0.9.1"
}

apply plugin: 'java'
apply plugin: 'application'

sourceCompatibility = '1.8'

mainClassName = 'plum.SchemaApp'

repositories {
    mavenCentral()
```

```

}

version = '0.1.0'

dependencies {
    compile 'org.apache.avro:avro:1.8.2'
}

jar {
    manifest {
        attributes 'Main-Class': mainClassName
    }

    from {
        configurations.compile.collect {
            it.isDirectory() ? it : zipTree(it)
        }
    } {
        exclude "META-INF/*.SF"
        exclude "META-INF/*.DSA"
        exclude "META-INF/*.RSA"
    }
}

```

- **1 A Gradle plugin to generate Java code from Apache Avro schemas**
- **2 The latest release of Avro at the time of writing**

Let's confirm that we can build this Gradle project:

```

$ gradle compileJava
...
BUILD SUCCESSFUL
...

```

Good—now we are ready to work on the schema for our health-check event.

6.2.2. Writing our health check event schema

To add our new Avro schema for the NCX-10 machine's health-check event, create a file at this path:

```
src/main/resources/avro/check.avsc
```

Populate this file with the Avro schema in the following listing. Note that we are using Avro's JSON-based schema syntax rather than the Avro IDL to model this event; this is what the `.avsc` file extension signifies.

Listing 6.2. `check.avsc`

```
{ "name": "Check",
  "namespace": "plum.avro",
  "type": "record",
  "fields": [
    { "name": "factory", "type": "string" },
    { "name": "serialNumber", "type": "string" },
    { "name": "status",
      "type": {
        "type": "enum", "namespace": "plum.avro", "name": "StatusEnum",
        "symbols": ["STARTING", "RUNNING", "SHUTTING_DOWN"]
      }
    },
    { "name": "lastStartedAt", "type": "long",
      "logicalType": "timestamp-millis" },
    { "name": "temperature", "type": "float" },
    { "name": "endOfLife", "type": "boolean" },
    { "name": "floorNumber", "type": ["null", "int"] }
  ]
}
```

This is a densely packed Avro schema file. Let's break it down: the top-level entity is a record called `Check`, which belongs in the `plum.avro` namespace (as would all of our entities). Our `Check` record contains seven fields, which correspond to the seven data points identified for a health-check event:

- The name of the factory in which the machine is installed is of type `string`.
- The machine's serial number is another `string`.
- The machine's current status is an Avro enum (short for *enumeration*), which can be one of `STARTING`, `RUNNING`, or `SHUTTING_DOWN`. An enum is a *complex type*, so we need to provide it with a namespace (`plum.avro`) and a name (`StatusEnum`).
- The time when the machine was last started, which is a Unix timestamp accurate to milliseconds. This is stored as a `long`, but Avro also lets us specify a *logical type* for the field, which gives a hint as to how a parser should handle the underlying type.
- The machine's current temperature, in Celsius, is a `float`.
- Whether or not the machine has been earmarked as being end-of-life is a `boolean`.

- The floor number is stored as a *union type* of an `int` or a `null`. If the factory does not have multiple floors, this field will be set to `null`. Otherwise, it will be an integer.

A minor piece of housekeeping—we need to soft-link the `resources/avro` subfolder to another location so that the Avro plugin for Gradle can find it:

```
$ ln -sr src/main/resources/avro src/main
```

With our Avro schema defined, let's use the Avro plugin in our `build.gradle` file to automatically generate Java bindings for our schema:

```
$ gradle generateAvroJava
:generateAvroProtocol UP-TO-DATE
:generateAvroJava

BUILD SUCCESSFUL

Total time: 8.234 secs
...
```

You will find the generated files inside your Gradle build folder:

```
$ ls build/generated-main-avro-java/plum/avro/
Check.java  StatusEnum.java
```

These files are too lengthy to reproduce here, but if you open them in your text editor, you will see that the files contain POJOs to represent the one record and the one enumeration that make up our health-check event.

So far, so good: we now have a model in Java for our Avro health check-event. In the next section, we will write simple Java code to work with these health checks.

6.2.3. From Avro to Java, and back again

Remember that Avro has two representations, a human-readable JSON encoding and a more efficient binary encoding. In this section, we will round-trip a health-check event from Avro's JSON-based encoding into a Java object, and then back again into Avro's binary encoding. This is not particularly useful for Plum, but it will give you a chance to become fa-

miliar with Avro's two representations and show you how to interact with Avro from a regular programming language such as Java.

Add the following code into a new file called `src/main/java/plum/AvroParser.java`.

Listing 6.3. AvroParser.java

```
package plum;

import java.io.*;
import java.util.*;
import java.util.Base64.Encoder;

import org.apache.avro.*;
import org.apache.avro.io.*;
import org.apache.avro.generic.GenericData;
import org.apache.avro.specific.*;

import plum.avro.Check;

public class AvroParser {

    private static Schema schema;
    static {
        try {
            schema = new Schema.Parser()
                .parse(AvroParser.class.getResourceAsStream("/avro/check.avsc"))
        } catch (IOException ioe) {
            throw new ExceptionInInitializerError(ioe);
        }
    }

    private static Encoder base64 = Base64.getEncoder();

    public static Optional<Check> fromJsonAvro(String event) {

        InputStream is = new ByteArrayInputStream(event.getBytes());
        DataInputStream din = new DataInputStream(is);

        try {
            Decoder decoder = DecoderFactory.get().jsonDecoder(schema, din);
            DatumReader<Check> reader = new SpecificDatumReader<Check>(schema);
            return Optional.of(reader.read(null, decoder));
        } catch (IOException | AvroTypeException e) {
            System.out.println("Error deserializing:" + e.getMessage());
            return Optional.empty();
        }
    }
}
```



```

public static Optional<String> toBase64(Check check) {

    ByteArrayOutputStream bout = new ByteArrayOutputStream();

    DatumWriter<Check> writer = new SpecificDatumWriter<Check>(schema);
    BinaryEncoder encoder = EncoderFactory.get().binaryEncoder(bout, nu
try {
    writer.write(check, encoder);
    encoder.flush();
    return Optional.of(base64.encodeToString(bout.toByteArray()));
} catch (IOException e) {
    System.out.println("Error serializing:" + e.getMessage());
    return Optional.empty();
}
}
}

```

- **1 Import the Check POJO autogenerated from the bundled schema.**
- **2 Statically initialize this Schema object from the bundled schema.**
- **3 Deserialize our event into a Check POJO, boxed in an Optional.**
- **4 Serialize our event in its binary format, and then Base64 encode it.**

The AvroParser file is simple; it consists of three parts:

- *Initialization code*—This gives us a reusable representation in Java of the Avro health-check schema; it also gives us a Base64 encoder, which we'll use later.
- *A static function, fromJsonAvro*—This converts an incoming event, which is hopefully a health check in Avro's JSON format, into a Check POJO. We box the return value in a Java 8 `Optional` to cover the case where the health check couldn't be correctly deserialized.
- *A static function, toBase64*—This converts our Check POJO into an Avro binary record, and then Base64 encodes that byte array to make it more human-readable. Again, we box the return value in an `Optional` to cover any serialization problems.

We can now stitch these two functions together via a new `SchemaApp` class containing our `main` method. Create a new file called `src/main/java/plum/SchemaApp.java` and populate it with the contents of the following listing.

Listing 6.4. SchemaApp.java

```

package plum;

```

```

import java.util.*;

import plum.avro.Check;

public class SchemaApp {

    public static void main(String[] args){
        String event = args[0];

        Optional<Check> maybeCheck = AvroParser.fromJsonAvro(event);

        maybeCheck.ifPresent(check -> {
            System.out.println("Deserialized check event:");
            System.out.println(check);

            Optional<String> maybeBase64 = AvroParser.toBase64(check);
            maybeBase64.ifPresent(base64 -> {
                System.out.println("Re-serialized check event in Base64:");
                System.out.println(base64);
            });
        });

    }
}

```

- **1 Attempt to deserialize our event into a Check POJO.**
- **2 Proceed only if we have a Check.**
- **3 Attempt to serialize our Check into a Base64-encoded byte string.**
- **4 Proceed only if we have a byte string.**

We will pass a single argument into our SchemaApp on the command line: a string hopefully containing a valid NCX-10 health-check event, in Avro's JSON format. Our code then attempts to deserialize this string into a check POJO. If this succeeds, we proceed to print out the Check and then convert it back into Avro's binary representation, Base64-encoded so that we can print it out easily.

Let's build our app now. From the project root, the plum folder, run this:

```

$ gradle jar
...
BUILD SUCCESSFUL

Total time: 25.532 secs

```

Great—we are now ready to test our new Avro-powered schema app.

6.2.4. Testing

Remember back in [section 6.1.1](#), where we introduced Plum and our NCX-10 health-check event, conveniently represented in JSON? Almost the same JSON is valid as an Avro representation of the same event. Here is the Avro representation:

```
{ "factory": "Factory A",
  "serialNumber": "EU3571",
  "status": "RUNNING",
  "lastStartedAt": 1539598697944,
  "temperature": 34.56,
  "endOfLife": false,
  "floorNumber": { "int": 2 }
}
```

The only difference is the `{ "int": ... }` syntax boxing the `floorNumber` property's value of 2. Avro requires this because the floor number in our health-check event is optional, and Avro represents optional fields as a *union* of null and the other type (in this case, an integer). We need to use the `{ "int": ... }` syntax to tell Avro that we want to treat the value of this union type as an integer, rather than a null.

Let's take this JSON representation of our health-check event and pass it into our freshly built Java app:

```
$ java -jar ./build/libs/plum-0.1.0.jar "{\"factory\":\"Factory A\",
  \"serialNumber\":\"EU3571\", \"status\":\"RUNNING\", \"lastStartedAt\":
  1539598697944, \"temperature\":34.56, \"endOfLife\":false,
  \"floorNumber\":{\"int\":2}}"
Deserialized check event:
{"factory": "Factory A", "serialNumber": "EU3571", "status": "RUNNING",
  "lastStartedAt": 1539598697944, "temperature": 34.56, "endOfLife": fal
  "floorNumber": 2}
Re-serialized check event in Base64:
EkZhY3RvcnkgQQxFVtM1NzEC3L2Zm+dVcT0KQgACBA==
```

Success! Our Java app has taken the incoming Avro event, successfully deserialized it into a Java POJO, printed that out, and then converted it back into a Base64-encoded binary string. Note how brief the Base64 string is. This representation is much more succinct than the JSON representation, for two reasons:

- The binary representation does not include property tags like `factory` or `status`. Instead, properties are identified positionally by using the associated schema.

- The binary representation can represent certain values efficiently—for example, using IDs for enum values, and 1 or 0 for Boolean true or false.

Before we move on, let's confirm that an invalid event fails processing:

```
$ java -jar ./build/libs/plum-0.1.0.jar "{\"factory\":\"Factory A\"}"
Error deserializing:Expected string. Got END_OBJECT
```

Good: the event correctly failed to deserialize, because various required properties are missing.

That completes our initial work with Avro for Plum. But the tight interdependency between Avro instances and their schemas raises interesting data-modeling questions, which we'll look at next.

6.3. Associating events with their schemas

Think about the preceding section: we deserialized an incoming Avro event in JSON format to a Java Check POJO, and then converted it back into a Base64-encoded version of that same event's binary encoding. But how did we know that the incoming Avro event was an NCX-10 health-check event? How did we know which schema to use to parse that incoming payload?

The simple answer is that we didn't. We assumed that the incoming string would be a valid NCX-10 health-check event, represented in Avro's JSON format. If that assumption was incorrect and the event did not match our Avro schema, we simply threw an error.

In this section, we will consider more-sophisticated strategies for associating events in a unified log with their schemas.

6.3.1. Some modest proposals

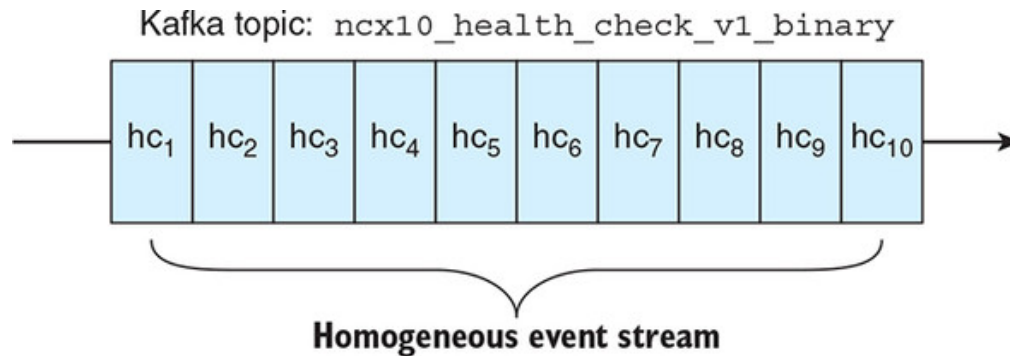
Let's imagine for a minute that we have implemented Apache Kafka as part of Plum's unified log. How do we write and deploy a Kafka worker that can successfully consume NCX-10 health-check events? Three potential strategies are described next.

One Kafka topic per event type (homogeneous streams)

In this model, Plum establishes a convention that each distinct event type will be written to its own Kafka topic. This needs to be specified down to the version of the schema and its format (binary or JSON). An example

name for this specific Kafka topic might be `ncx10_health_check_v1_binary`, as illustrated in [figure 6.3](#).

Figure 6.3. A homogeneous Kafka topic, containing only events in the Avro binary representation of version 1 of the NCX-10 health check. A Kafka worker can consume this safely in the knowledge that all records in this topic can be deserialized using the specific Avro schema.



The merit of this approach is its simplicity: the Kafka topic is completely homogeneous, containing only NCX-10 health checks. But as a result, we will end up proliferating event streams at Plum: one event stream (aka Kafka topic) for each version of each schema.

Therefore, if we want to do something as simple as parse multiple versions of the NCX-10 health-check event, we would have to write a stateful stream-processing app that unites data across multiple Kafka topics, one per schema version. And an analysis across all of our event types (for example, to count events per hour) is even more painful: we have to join each and every Kafka topic together in a stateful stream processor to achieve this.

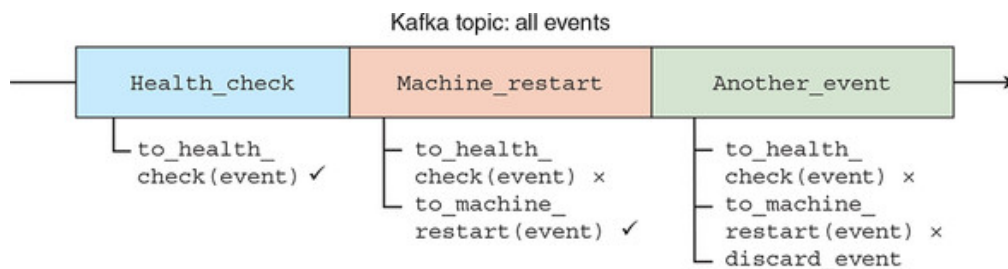
Heterogeneous streams that contain multiple event types are much easier to work with. But how do we know which events they contain?

Trial-and-error deserialization (heterogeneous streams)

You could call this the *brute-force* approach: Plum would mix lots of event types into one stream, and then workers would attempt to deserialize events into any given schema that they are interested in. If the event fails to deserialize into the appropriate POJO, the worker ignores that event.

[Figure 6.4](#) shows this approach.

Figure 6.4. Our consuming application is interested in only two event types (*health check* and *machine restart*) and uses a trial-and-error approach to identify these events in the Kafka topic. The amount of wasteful deserialization increases with the number of event types that our consuming app is concerned with.



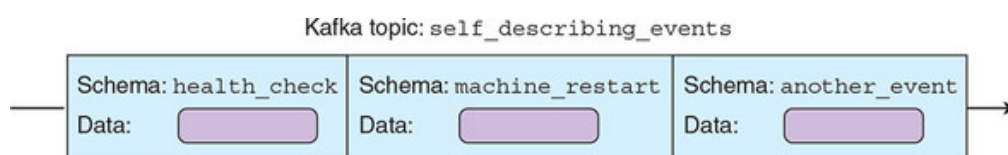
As soon as we go beyond toy applications, this approach is incredibly inefficient. Imagine that we have an application that needs to work on five event types. Deserialization is a computationally expensive task, and our application will attempt to deserialize each event up to five times before moving on to the next event.

There has to be a better way than this brute-force approach. We cover this next.

Self-describing events (heterogeneous or homogeneous streams)

In this approach, we again mix many event types into a single stream. The difference this time is that each event has a simple metadata “envelope” attached to it, which tells any consuming application which schema was used to serialize the event. We can call these events *self-describing events*, because the event carries with it at all times the information about what schema this instance is associated with. [Figure 6.5](#) depicts this approach.

Figure 6.5. In this Kafka topic, we can see three self-describing events: a health check, a machine restart, and another event. Each event consists of a schema describing the event, plus the event itself.



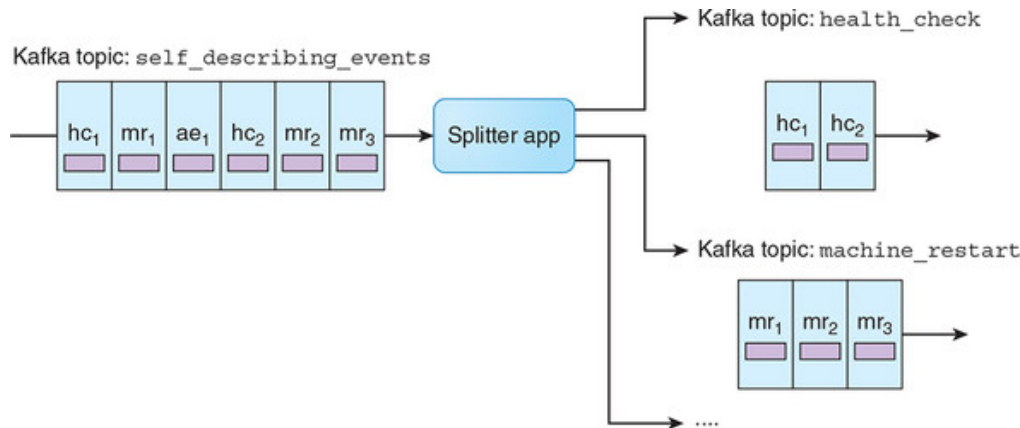
Working with self-describing events is a two-step process:

1. Parse the event’s metadata envelope to retrieve the identifier for the event’s schema.
2. Parse the event’s data portion against the identified schema.

This is a powerful approach: for the cost of a little more deserialization work, we now have a much more flexible way of defining our events. We can write these events into a single heterogeneous stream, but equally we can then “shred” that stream into substreams of single or associated event types. Because the event’s schema travels with the event itself, we

can send the event on anywhere without losing the necessary metadata to parse the event. This is visualized in [figure 6.6](#).

Figure 6.6. With self-describing events, we can switch between heterogeneous streams and homogeneous streams at will, because a reference to the schema travels with the event. In this example, we have a “splitter app” separating *health checks* and *machine restarts* into dedicated Kafka topics.



An important thing to note: with self-describing events, we are always talking about adding some kind of *pointer* to the event’s original schema to the event. This pointer is a reference rather than the schema itself, not least because the schema itself is often huge—larger even than the event payload!

Let’s build a self-describing event for Plum in the next section.

6.3.2. A self-describing event for Plum

How should we represent a self-describing event in Apache Avro? We need a metadata “envelope” to wrap the event and record which schema was used to serialize the event. It makes sense for us to define this in Avro itself, and we’ve set out a simple proposal in the following listing.

Listing 6.5. `self_describing.avsc`

```
{ "name": "SelfDescribing",
  "namespace": "plum.avro",
  "type": "record",
  "fields": [
    { "name": "schema", "type": "string" },
    { "name": "data", "type": "bytes" }
  ]
}
```

You can see two fields in the `SelfDescribing` record:

- `schema` is a string to identify the schema for the given event.
- `data` is a sequence of 8-bit unsigned bytes, itself containing the Avro binary representation of the given event.

How would we identify the schema in a string format? Let's use something simple, such as this:

```
{vendor}/{name}/{version}
```

This format is a simplified implementation of the schema URIs we use in our Iglu schema registry system at Snowplow (<https://github.com/snowplow/iglu>). In this implementation are the following:

- `vendor` tells us which company (or team within the company) authored this schema. This can be helpful for telling us where to find the schema, and helps prevent naming conflicts (for example, if Plum and one of its software partners both define an ad click event).
- `name` is the name of the event.
- `version` gives the version of the given event. For simplicity at Plum, we will use an incrementing integer for the version.

In the case of the initial version of our NCX-10 health-check event, the schema string would look like this:

```
com.plum/ncx10-health-check/1
```

This self-describing envelope is a good fit for Avro's binary representation. But an interesting dilemma crops up when we think about Avro's JSON-based format. We could use the same approach, in which case a self-describing health-check event would look like this:

```
{ "schema": "com.plum/ncx10-health-check/1",
  "data": <<BYTE ARRAY>>
}
```

But there isn't a lot of point in this format: it's less compact than the binary representation, the data portion is itself still in the binary representation, and it's still not human-readable. It has little going for it! A better implementation of self-describing Avro for the JSON format would look something like this:

```
{ "schema": "com.plum/ncx10-health-check/1",
  "data": {
```



```

    "factory": "Factory A",
    "serialNumber": "EU3571",
    "status": "RUNNING",
    "lastStartedAt": 1539598697944,
    "temperature": 34.56,
    "endOfLife": false,
    "floorNumber": { "int": 2 }
  }
}

```

This is better: it's significantly less compact than the binary representation, but it is human-readable from top to bottom. The only oddity is that the overall payload is in fact no longer an Avro: it is valid JSON, and the data portion is a valid JSON-format Avro, but the overall payload is *not* an Avro. This is because there is no Avro schema that models the entire payload; instead to process an instance of this event, we would do the following:

1. Parse the event initially as JSON to extract the schema string.
2. Retrieve the Avro schema per the schema string.
3. Use the schema string to retrieve the JSON node for the event's data.
4. Deserialize the JSON data node into a Check POJO object by using the retrieved Avro schema.

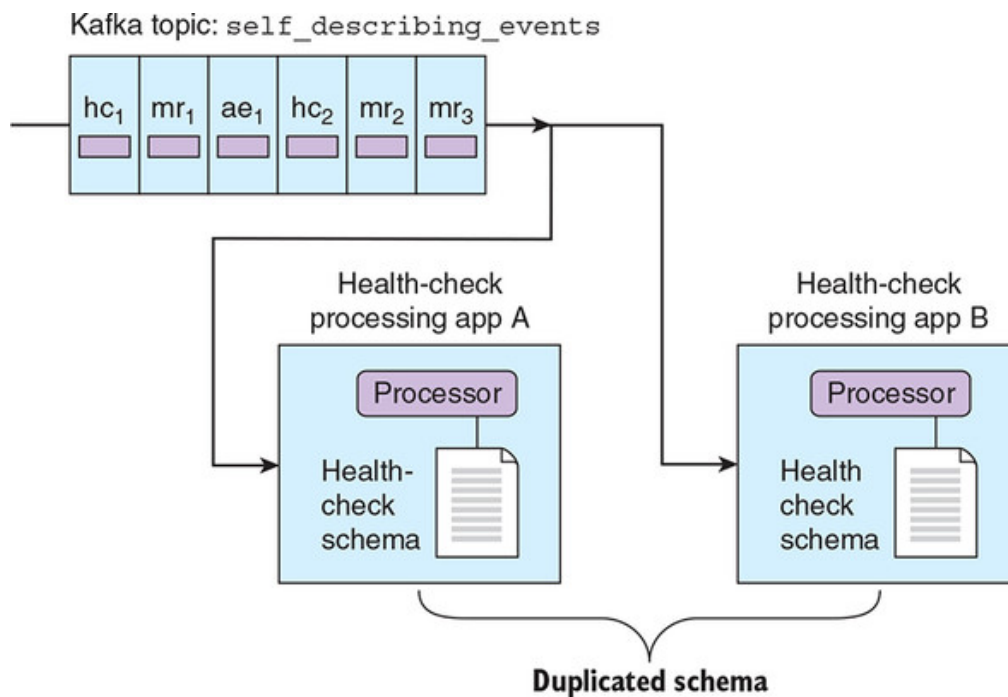
Don't worry if the nuances of self-describing Avro seem abstract right now. We will put these ideas into practice in the next chapter; things should soon seem more concrete.

Before we move on, let's find a home for all of our schemas: a schema registry.

6.3.3. Plum's schema registry

In the preceding example, we defined a health-check event for the NCX-10 machine by using the Avro schema language, and then embedded this Avro definition inside our Java application. If we took this to its logical conclusion, we would be embedding this Avro definition file into every app that needed to understand NCX-10 health-check events. [Figure 6.7](#) illustrates this process.

Figure 6.7. We have two Kafka applications that want to process health-check events. Both of these apps contain a definition of the health-check event in Avro; there is no central source of truth for this schema.



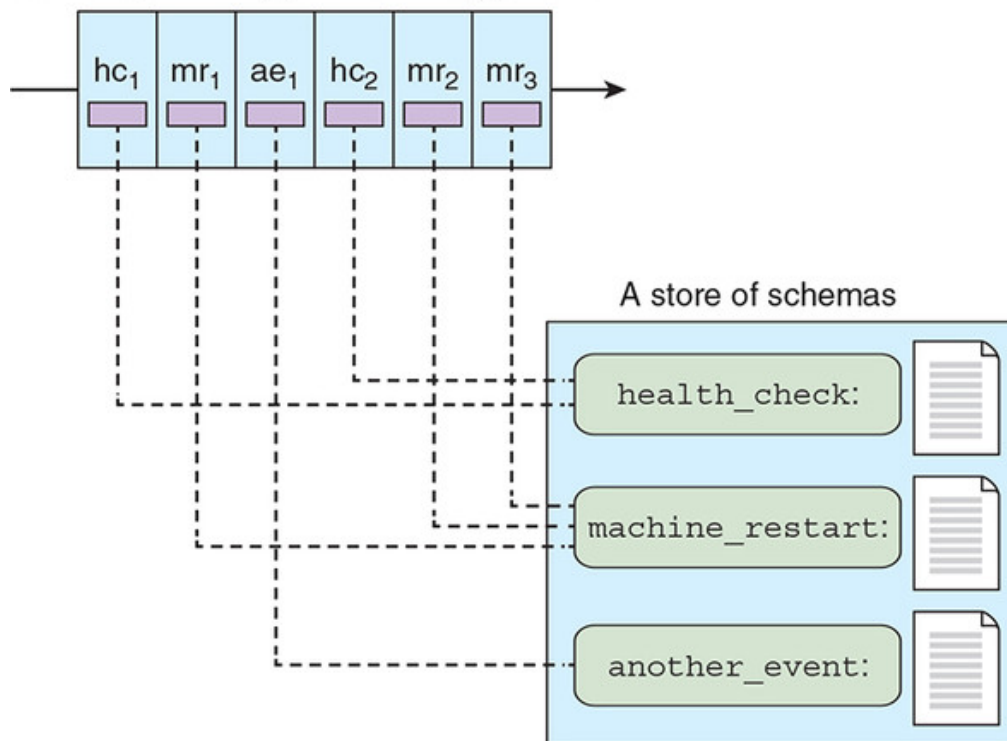
This copy-and-paste approach is a bad idea, because we don't have a *single source of truth* for the definition of an NCX-10 health-check event within Plum. Instead, we have multiple definitions of this event, scattered around multiple codebases. We have to hope that all the definitions are the same, or Plum will experience the following:

- Self-describing events being written to Kafka with contradictory data structures for the same schema
- Runtime failures in our Kafka consuming applications as they expect one event structure and get another

Plum needs a single source of truth for all of our schemas—a single location that registers our schemas, and that all teams can access to understand the definition of each schema. [Figure 6.8](#) visualizes the schema registry for Plum, containing three event schemas.

Figure 6.8. Plum now has a schema registry that contains the master copy of the schema for each type of event. All consuming and producing apps should use these master copies.

Kafka topic: self_describing_events



At its simplest, a schema registry can be just a shared folder, perhaps on S3, HDFS, or NFS. The schema syntax we defined earlier maps nicely onto a folder structure, here in Amazon S3:

```
s3://plum-schemas/com.plum/ncx10-health-check/1
```

The file at the preceding path would be the Avro definition file for version 1 of the NCX-10 health-check event.

Going beyond a simple shared folder structure, there are two actively developed open source schema registries:

- *Confluent Schema Registry* (<https://github.com/confluentinc/schema-registry>)—An integral part of the Confluent Platform for Kafka-based data pipelines. Confluent Schema Registry supports Avro only, with first-class support for Avro's schema evolution. It uses Kafka as the underlying storage mechanism and is a distributed system with a single master architecture. It assigns a registry-unique ID (monotonically increasing) to each registered schema.
- *Iglu* (<https://github.com/snowplow/iglu>)—An integral part of the Snowplow open source event data pipeline. Iglu supports multiple schema technologies, including Avro, JSON Schema, and Thrift. It supports schema resolution across multiple schema registries and uses semantic URIs to address schemas. Iglu is used in Snowplow but intended to be general-purpose (with Scala and Objective-C client libraries).

Putting either of these schema registries into service for Plum is beyond the scope of this book, but we encourage you to check them out.

Summary

- A unified log is a *decoupled* architecture: consumers and producers of event streams have no particular knowledge of each other.
- The contract between event consumers and producers is represented by the schema of each event.
- Schema technologies we can use include JSON Schema, Apache Avro, Thrift, and protocol buffers. Avro is a good choice.
- We can define our event schemas by using Avro's JSON-based schema syntax, and then use code-generation to build Java bindings (think PO-JOs) to the schemas.
- Avro has a JSON representation for data, and a binary representation. We wrote a simple Java app to convert the JSON representation into Java and back into binary.
- We need a way to associate events with their schemas: either one stream per schema, trial and error, or self-describing events.
- We can model a simple self-describing event for Avro by using Avro itself. We need slightly different implementations for the binary- and JSON-based representations.
- A schema registry is a central repository for all of our schemas: this is the source of truth for which schemas are available as part of our company's unified log.

[Support](#) [Sign Out](#)